

Neo4j CTI DB Schema for IFT Dataset Generation

Date: 02/13/2026

Author: Ionut Cardei

Change Log:

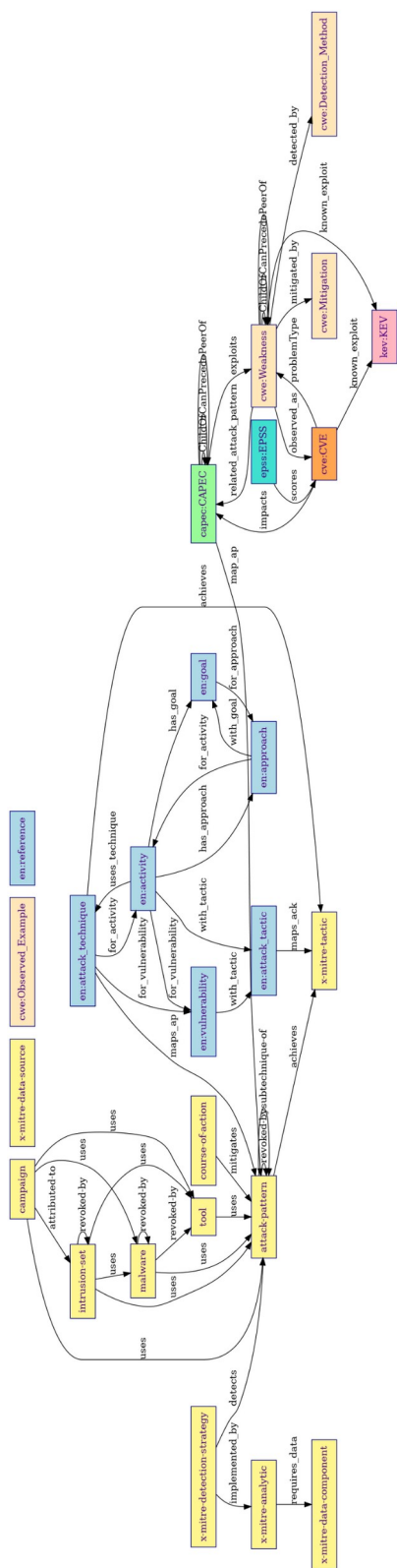
02/13/26	• add: relationship from CAPEC - [:map_ap] -> attack-pattern using Attack_Pattern/Taxonomy_Mappings XML element • add: ATT&CK node x-mitre-data-source and relationship x-mitre-data-component - [:belongs_to] -> x-mitre-data-source • change: CAPEC property example_instances type changed to list[string]o • add: CAPEC prperty execution_flow of type JSON string • add: CVE property affected of type JSON string • add: CVE property metrics of type JSON string • change: CVE property name cvss3_1_vectorString • add: CVE property name cvss3_1_baseScore • add: CVE property name cvss3_1_attackVector • add: CWE Weakness property extended_description • add: CWE Weakness property Modes_Of_Introduction, list[string] • change: CWE Weakness property functional_areas type to list[string] • change: CWE Weakness property name to likelihood_of_exploit • change: CWE Mitigation property names should start with lower case • change: CWE Detection_Method property names should start with lower case • change: CWE observed_as edge now has edge properties
2/16/2026	• change: CAPEC - EXPLOITS - Weakness edge name to 'exploits' • add: KEV schema • add: CVE - [:known_exploit] -> KEV • add: cwe:Weakness - [:known_exploit] -> KEV • change: edge name 'maps_ack' to 'maps_ap' in ENGAGE:attack_technique - maps_ack - ATTACK:attack-pattern
2/20/2026	• change: attack-pattern detection: add x-mitre-detection-strategy and x-mitre-analytic nodes and new relationships
3/12/2026	• corrected the CVE node property list (summary)
3/19/2023	• CAPEC node: added abstraction property to the summary
3/20/2023	• Added EPSS schema info

Status: ATT&CK, CWE, CVE, CAPEC, ENGAGE, KEV, EPSS included.

This document summarizes the schema needed for IFT dataset generation, with information taken from the IFT Design Document and from source CTI documents.

The motivation for this file is to assist the development work needed to import CTI data into the neo4j DB with the original rich and descriptive schema that CTI source documents already posses.

[schema graph page]



Neo4J Concept Naming Constraint

Neo4J requires node/edge names and, possibly, property names inside Cypher query strings that contain special characters, like '-', to be escaped with backquote signs. Note that underscore '_' is acceptable inside an identifier. Identifiers with '-' are common in ATT&CK. e.g. **attack-pattern** must be written as ``attack-pattern`` in a Cypher query.

The following function from file [tmpl_gen/src/tmpl_gen/neo4j_utils.py](#) in Athena [Github repository Sophia-tmpl_gen](#) does that quotation:

```
def neo4j_safe_identif(name:str) -> str:
    return name if name.isidentifier() else '`' + name + '`
```

With this, we could keep the original node and relationship names from the ATT&CK specification, instead of renaming them.

Node / Edge Property Types

Neo4j supports all types listed below, except for dictionaries, which need to be stored in JSON string format. Type list indicates a list[str|dict].

Date/time property values are represented as strings in the JSON source documents. They must be converted to **datetime** objects.

Function **convert_prop_cypher** in file [tmpl_gen/src/tmpl_gen/neo4j_utils.py](#) in Athena [Github repository Sophia-tmpl_gen](#) does type conversion for property values included in Cypher queries:

```
def convert_prop_cypher(parsecfg:dict, propname:str, propval):
    datetime_props = parsecfg.get("datetime_properties", [])
    val = propval
    match propval:
        case str() if propname in datetime_props:
            val = dateutil.parser.parse(propval)
        case list() if not all(isinstance(p, str) for p in propval):
            val = str(propval)
        case dict():
            val = json.dumps(propval, indent=4)
        case _:
            pass
    return val
```

Consistent Concept IDs

It is imperative that the ID property of all CTI concepts be imported correctly, since records frequently cross reference concepts across different domains using concept IDs. E.g. a CWE record links to CVE records as observed examples using the **cveId**.

A uniform ID scheme internal to CTI DB is already in place, with the id field called **stix_id**.

ATT&CK CTI DB Schema

The node, property, and relationship details are given in a terse, computer-readable format that makes it easy for developers to copy-paste into Python code, as needed.

This is the mapping for property types used for ATT&CK.

```
0: str
1: bool
2: datetime
3: list
4: json
```

Note that some properties may not exist. This is true at least for CVE records. Create properties with a default standard value “N/A” or “none” to allow Cypher queries to proceed without failing.

Nodes are described in this format:

```
nodename: alias
properties: prop1, prop2, ...
```

The alias can be used as the official node name when added to the neo4j DB. Unless there is a good reason (e.g. compatibility with other tools) to rename the node names when inserted to the neo4j DB I recommend we stick to their original MITRE names.

The Sophia template generation code already supports defining user-friendly aliases for nodes/relationship/property names.

Property type matching

This is the mapping from node/edge property names in ATT&CK to their neo4j property type:

```
aliases/3, created/2, description/0, external_references/4, first_seen/2, id/0,
is_family/1, kill_chain_phases/4, last_seen/2, mitre_id/0, modified/2, name/0,
revoked/1, type/0, x_mitre_aliases/3, x_mitre_data_sources/3, x_mitre_deprecated/1,
x_mitre_detection/0, x_mitre_domains/3, x_mitre_platforms/3, x_mitre_is_subtechnique/1,
x_mitre_shortcode/0, x_mitre_version/0, x_mitre_analytic_refs/3,
x_mitre_log_source_references/4, x_mitre_mutable_elements/4
```

Nodes and Properties

Format for node first line:

```
nodename: alias
```

NOTE: alias is commonly used in technical docs and our templates.

Node aliasing is done by tmpl_gen and should be ignored by the CTI DB.

```
attack-pattern: technique
properties: created, description, external_references, id, kill_chain_phases, mitre_id,
modified, name, type, x_mitre_data_sources, x_mitre_deprecated, x_mitre_detection,
x_mitre_domains, x_mitre_is_subtechnique, x_mitre_platforms, x_mitre_version,
x_mitre_attack_spec_version
```

```
campaign: none
properties: aliases, created, description, external_references, first_seen, id,
last_seen, mitre_id, modified, name, revoked, type, x_mitre_deprecated, x_mitre_domains,
x_mitre_version, x_mitre_attack_spec_version
```

```
course-of-action: mitigation
```

properties: created, description, external_references, id, mitre_id, modified, name, spec_version, type, x_mitre_attack_spec_version, x_mitre_deprecated, x_mitre_domains, x_mitre_modified_by_ref, x_mitre_version, x_mitre_attack_spec_version

intrusion-set: none

properties: aliases, created, description, external_references, id, mitre_id, modified, name, revoked, type, x_mitre_deprecated, x_mitre_domains, x_mitre_version, x_mitre_attack_spec_version

malware: none

properties: created, description, external_references, id, is_family, mitre_id, modified, name, revoked, type, x_mitre_aliases, x_mitre_deprecated, x_mitre_domains, x_mitre_platforms, x_mitre_version, x_mitre_attack_spec_version

tool: none

properties: created, created_by_ref, description, external_references, id, mitre_id, modified, name, revoked, type, x_mitre_aliases, x_mitre_deprecated, x_mitre_domains, x_mitre_platforms, x_mitre_version

x-mitre-analytic: analytic

properties: created, created_by_ref, description, external_references, id, mitre_id, modified, name, type, x_mitre_deprecated, x_mitre_domains, x_mitre_platforms, x_mitre_version, x_mitre_attack_spec_version, x_mitre_log_source_references, x_mitre_mutable_elements

NOTE: property x_mitre_log_source_references[i]['x_mitre_data_component_ref'] refers to data component nodes, defining relationship x-mitre-analytic - x-mitre-data-component

x-mitre-data-component: data-component

properties: created, description, id, mitre_id, modified, name, revoked, type, x_mitre_deprecated, x_mitre_domains, x_mitre_version

x-mitre-data-source: data-source

properties: created, description, id, mitre_id, modified, name, revoked, type, x_mitre_deprecated, x_mitre_domains, x_mitre_version, x_mitre_attack_spec_version

IMPLEMENTATION NOTE: relationship x-mitre-data-component - belongs_to - x-mitre-data-source relationship is described by property x-mitre-data-component.x_mitre_data_source_ref that is the STIX ID of the parent data source node.

x-mitre-detection-strategy: detection-strategy

properties: created, external_references, mitre_id, modified, name, type, x_mitre_deprecated, x_mitre_domains, x_mitre_analytic_refs, x_mitre_version, x_mitre_attack_spec_version

NOTE: property x-mitre-detection-strategy.x_mitre_analytic_refs is a list of x-mitre-analytic IDs that should be used to define the **x-mitre-detection-strategy - implemented_by - x-mitre-analytic** relationship.

x-mitre-tactic: tactic

properties: created, description, external_references, id, mitre_id, modified, name, type, x_mitre_deprecated, x_mitre_domains, x_mitre_shortcode, x_mitre_version, x_mitre_attack_spec_version

ATT&CK Edges

Edge Property Types

This is a repeat of the same type map:

```
0: str
1: bool
2: datetime
3: list
4: json
```

All relationships have the same properties and the following subset with their type mapping need to be imported:

```
created/2, description/0, modified/2, relationship_type/0, x_mitre_deprecated/1
```

This is the list of relationships:

```
attack-pattern revoked-by attack-pattern
attack-pattern subtechnique-of attack-pattern
campaign attributed-to intrusion-set
campaign uses attack-pattern
campaign uses malware
campaign uses tool
course-of-action mitigates attack-pattern
intrusion-set revoked-by intrusion-set
intrusion-set uses attack-pattern
intrusion-set uses malware
intrusion-set uses tool
malware revoked-by malware
malware revoked-by tool
malware uses attack-pattern
tool uses attack-pattern
```

```
x-mitre-data-component detects attack-pattern
```

```
x-mitre-data-component belongs_to x-mitre-data-source
```

NOTE: add no properties

```
x-mitre-detection-strategy implemented_by x-mitre-analytic
```

```
x-mitre-detection-strategy detects attack-pattern
```

NOTE: add no properties

Unrepresented Relationship:

attack-pattern achieves x-mitre-tactic

Each attack-pattern points to the tactic it achieves. This very important relationship between attack-pattern and x-mitre-tactic is not represented explicitly in the MITRE AT&CK documents. Adding this relationship to the CTI DB requires custom coding.

The relationship is identified as following:

An attack-pattern has property “kill_chain_phases” that is a list of dictionaries, as in this example:

```
"kill_chain_phases": [
  {
    "kill_chain_name": "mitre-attack",
    "phase_name": "credential-access"
  },
  {
    "kill_chain_name": "mitre-attack",
    "phase_name": "collection"
  }
]
```

The "kill_chain_name" fields are always "mitre-attack".

The "phase_name" fields are short names of the tactics achieved by the current technique.

An x-mitre-tactic's short name is defined by the value of property "x_mitre_shortcode".

Code to parse this relationship from the enterprise-attack.json file can be found in function

parse_custom_rels from file ***tmpl_gen/src/tmpl_gen/utils.py*** in Athena Github repository ***Sophia_tmpl_gen***.

A custom entry in the parsecfg_attack["custom_relparsers"] list provides the node names and paths for function ***parse_custom_rels*** to generate this relationship. Similar code can be used for custom import code in other JSON sources, such as CVE.

CWE CTI DB Schema

To find the node in XML sources, such as CWE, a path in the XML tree is specified.

The notation of a path for an element in a list is the following. '0' stands for an index ≥ 0 .

Weakness_Catalog/Weaknesses/Weakness/0

All list elements with an index of 0 should be imported as a Cypher list.

Notation for a path relative to the current node:

.../Alternate_Terms

Notation for an XML element attribute: @ID. The use of @ sign is to help us distinguish from a node or property name.

Notation for child XML elements that should be converted to **a list property**, with path relative to the current node path:

.../Alternate_Terms/0/Alternate_Terms/Term

Alternate_Terms should be the property name, a list built from Term nodes content. '0' indicates the index position in the path, as Alternate_Terms is a list element. An index of -1 indicates that only the last list element must be imported – useful for keeping only the last *Modified* date.

CWE Nodes and Properties

Next is the list of nodes and their properties.

Node: Weakness

Path: Weakness_Catalog/Weaknesses/Weakness/0

Properties:

.../@ID	→ rename to id	string, CWE record ID
.../@Name	→ rename to name	string, weakness name
.../@Diagram	→ rename to diagram	string, path or URL to diagram image
.../@Abstraction	→ rename to abstraction	string

.../Content_History/Modification/-1/Modification_Date

```
→ rename to modified                               datetime
.../Description → rename to description             string, short description text
.../Extended_Description → rename to extended_description string, long description
.../Functional_Areas
→ rename to functional_areas list[string], e.g. "Memory Management", "Logging"
.../Likelihood_Of_Exploit → rename to likelihood_of_exploit string
.../Modes_Of_Introduction → rename it modes_of_introduction
Extract .../Modes_Of_Introduction/Introduction/0/Phase → rename to phases list[string]
```

Node: Detection_Method

Properties:

NOTE: the same Detection_Method node may be defined by more than one Weakness. Import into CTI DB only once and create edge Weakness - [:detected_by] -> Detection_Method. Use Detection_Method.Detection_Method_ID to identify nodes.

Node: Observed_Example

Properties:

```
summary of properties:
reference, link, description
```

Properties:

summary of properties:
id, description, effectiveness, phase, strategy

CWE Edges

Multiple Edge types: Weakness - [:X] -> Weakness, for X in "ChildOf", "CanPrecede", "PeerOf"

Path: Weakness_Catalog/Weaknesses/Weakness/0/Related_Weaknesses/Related_Weakness/0

Source node: current Weakness

Target node: Weakness with ID given by attribute @CWE_ID (property "id")

Edge Type given by attribute @Nature: one of "ChildOf", "CanPrecede", "PeerOf"

Edge: Weakness - [:related_attack_pattern] → CAPEC

Path:

Weakness_Catalog/Weaknesses/Weakness/0/Related_Attack_Patterns/Related_Attack_Pattern/0

Source node: current Weakness

Target node: CAPEC node with ID given by attribute @CAPEC_ID

Edge Type: related_attack_pattern

Edge: Weakness - [:mitigated_by] -> Mitigation

Path: Weakness_Catalog/Weaknesses/Weakness/0/Potential_Mitigations/Mitigation/0

Source node: current Weakness

Target node: Mitigation node with ID @Mitigation_ID

Edge Type: mitigated_by

Edge: Weakness - [:detected_by] -> Detection_Method

Path: Weakness_Catalog/Weaknesses/Weakness/0/Detection_Methods/Detection_Method/0

Source node: current Weakness

Target node: Detection_Method node with ID @Detection_Method_ID

Edge Type: detected_by

Edge: Weakness - [:observed_as] -> CVE

Path: Weakness_Catalog/Weaknesses/Weakness/0/Observed_Examples/Observed_Example/0

Source node: current Weakness

Target node: CVE node with CVE ID in the <Reference> node

Edge Type: observed_as

Edge Properties:

.../Description → rename to description string

.../Link → rename to link string

Note: these are XML element strings, not attributes

Edge: Weakness - [:known_exploit] -> KEV

Defined defined by KEV.cwes ID list property, multiplicity *.

CVE Schema

CVE records are specified in JSON format, it seems one file per CVE record.

Note that some properties could be missing. Use the standard value for these.

CVE Nodes and Properties

Node: CVE

Properties:

cveMetadata/cveId → rename to id string, assigned by CNA

cveMetadata/datePublished → rename to created datetime

cveMetadata/dateUpdated → rename to modified datetime

containers/cna/title string

affected:

Path: containers/cna/affected JSON string

containers/cna/affected/0/product → rename to affected_products list[str]

containers/cna/descriptions/0/value → rename to descriptions list[str]

metrics:

Path: containers/cna/metrics JSON string

containers/cna/metrics/0/cvssV3_1/vectorString → rename to cvss3_1_vectorString string

containers/cna/metrics/0/cvssV3_1/attackVector → rename to cvss3_1_attackVector string

containers/cna/metrics/0/cvssV3_1/baseScore → rename to cvss3_1_baseScore float

containers/cna/workarounds/0/description → rename to workarounds list[str]

containers/cna/exploits/0/description → rename to exploits list[str]

summary of properties:

id, created, modified, title, affected, affected_products, descriptions, metrics, cvss3_1_vectorString, cvss3_1_baseScore, cvss3_1_attackVector, workarounds, exploits

CVE Edges

Edge: CVE - [:impacts] -> CAPEC

Path: containers/cna/impacts/[]/capecId string, ID of CAPEC record

Edge: CVE - [:problemType] -> CWE

Path: containers/cna/problemTypes/[]/descriptions/[]/cweId string, CWE record ID

Edge: CVE - [:known_exploit] -> KEV

Defined defined by KEV.cveId property, multiplicity 1.

CAPEC Schema

The CAPEC XML schema is described on [this page](#), last modified in 01/2023.

CAPEC Nodes and Properties

Node: CAPEC

Path: Attack_Pattern_Catalog/Attack_Patterns/Attack_Pattern/0

Properties:

.../@Abstraction	→ rename to abstraction	string, "Detailed" or "Standard" or "Meta"
.../@ID	→ rename to id	int, CAPEC ID
.../@Name	→ rename to name	string, attack pattern name
.../@Status	→ rename to status	string, "Stable" or "Draft"

Property: consequences

Type: list[str]

Path: .../Consequences/Consequence/0

Special formatting required.

Notation below: {x} represents the string in <x>string</x> or an attribute @x

Subelements for a <Consequence> are:

.../Impact	string, e.g. Execute Unauthorized Commands
.../Scope	string, e.g. Availability, Confidentiality, Integrity
.../Note	string

Format for consequences[i]: "Scope: {Scope1}, {Scope2},...; Impact: {Impact}"

Property: created

Type: datetime

Path: .../Content_History/Submission/Submission_Date/

.../Indicators/0/Indicator → rename to indicators list[string]

Property: modified

Type: datetime

Path: .../Content_History/Modification/-1/ Modification_Date

.../Description → rename to description string

Property: example_instances

Type: list[string]

Path: .../Example_Instances/0/Example

.../Likelihood_Of_Attack → rename to likelihood_of_attack string, e.g. "Low", "High"

.../Mitigations/0/Mitigation → rename to mitigations list[string]

.../Prerequisites/0/Prerequisite → rename to prerequisites list[string]

Property: skills_required
 Type: list[str]
 Path: .../Skills_Required/0
 Special formatting required.
 Notation below: {x} represents the string in <x>string</x> or an attribute @x
 Format for skills_required[i]: "Level: {Skill@Level}. {Skill}"

.../Typical_Severity → rename to severity string, e.g. "Low", "Medium", ...

Property: execution_flow
 Type: JSON string
 Path: .../Execution_Flow
Implementation note: may convert Execution_Flow XML tree string to JSON string using
 xmltodict.parse()

summary of properties:
 abstraction, id, name, status, consequences, created, modified, description,
 example_instances, likelihood_of_attack, mitigations, prerequisites, skills_required,
 severity, execution_flow

CAPEC Edges

CAPEC - [:nature] -> CAPEC

Path: .../Related_Attack_Patterns/0/Related_Attack_Pattern/
 Target node identified by @CAPEC_ID attribute
 @Nature attribute, string, relationship type: ChildOf, CanPrecede, PeerOf

CAPEC - [:exploits] -> CWE

Path: .../Related_Weaknesses/Related_Weakness/0
 Target node identified by @CWE_ID attribute

CAPEC - [:map_ap] -> attack:attack-pattern

Implementation note: Identify numeric technique ID using
 Attack_Pattern/Taxonomy_Mappings/Entry_ID XML element for Taxonomy_Name="ATTACK".
 There may be multiple mappings to attack patterns:

```
<Taxonomy_Mappings>
  <Taxonomy_Mapping Taxonomy_Name="ATTACK">
    <Entry_ID>1498.001</Entry_ID>
    ...
  </Taxonomy_Mapping>
  ...
</Taxonomy_Mappings>
```

ENGAGE Schema

The ENGAGE dataset is packaged in several JSON files. Relationships between ENGAGE nodes are derived from object keys being. Nodes refer to ATT&CK attack-patterns (techniques) and tactics.

ENGAGE Nodes and Properties

Node: goal

Properties:
id, name, description, long_description, type

Node: activity

Properties:
id, name, description, long_description, type

Node: approach

Properties:
id, name, description, long_description, type

Node: vulnerability

Properties:
id, eav

Node: attack_technique

Properties:
id, name

Node: attack_tactic

Properties:
id, name

Node: reference

Properties:
id, title, url

ENGAGE Edges

Relationship	File name	Comment
activity -[:has_goal]-> goal	activity_details.jsonl	
activity -[:has_approach]-> approach	activity_details.jsonl	
activity -[:for_vulnerability]-> vulnerability	activity_details.jsonl	
activity -[:uses_technique]-> attack_technique	activity_details.jsonl	indirect to ATT&CK technique
activity -[:with_tactic]-> attack_tactic	activity_details.jsonl	
activity -[:has_reference]-> reference	activity_details.jsonl	
approach -[:with_goal]-> goal	approach_details.jsonl	
approach -[:for_activity]-> activity	approach_details.jsonl	
goal -[:for_approach]-> approach	goal_approach_mappings.jsonl, goal_details.jsonl	
attack_technique -[:maps_ap]-> ATTACK:attack-pattern	activity_details.jsonl, attack_mapping.jsonl	

```

attack_technique -[:for_vulnerability]-> vulnerability    attack_mapping.jsonl
attack_technique -[:for_activity]-> activity                attack_mapping.jsonl
attack_technique -[:achieves]-> ATTACK:x-mitre-tactic
                                                         attack_tactics_techniques.jsonl

attack_tactic -[:maps_ack]-> ATTACK:x-mitre-tactic activity_details.jsonl

vulnerability -[:with_tactic]-> attack_tactic activity_details.jsonl  vul. as
key in act. dct.
reference -[:refers]-> activity

```

KEV Schema

The KEV dataset is available at <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>. All KEVs are packaged in [one JSON file](#). The JSON schema is described [here](#).

KEV Nodes and Properties

Node: KEV

```

Properties:
cveID:                string
vendorProject:        string
product:              string
vulnerabilityName:    string
dateAdded:            datetime
shortDescription:     string
requiredAction:       string
dueDate:              datetime
knownRansomwareCampaignUse: string, 'Known'/'Unknown'
notes:               string
cwes:                list[string]

```

summary of properties: cveID, vendorProject, product, vulnerabilityName, dateAdded, shortDescription, requiredAction, dueDate, knownRansomwareCampaignUse, notes, cwes

KEV Edges

```

KEV - [:for_CVE] -> CVE          defined by KEV.cveId property, multiplicity 1

```

EPSS Schema

The Exploit Prediction Scoring System ([EPSS](#)) is a data-driven machine-learning model that estimates the probability that a published CVE will be exploited in the wild in the next 30 days.

It replaces subjective severity judgments with empirical signals from observed exploitation and ongoing activity, helping you focus limited remediation effort where attacks are most likely. EPSS publishes a 0–1 probability (with ranking percentiles) every day for every CVE and makes the data freely and openly accessible via CSV and API, so it slots easily into workflows and dashboards.

The EPSS [schema](#) and [dataset](#) (CSV file).

EPSS Nodes and Properties

Node: EPSS

Properties:

cve	string	CVE ID
epss	float [0,1]	probability of exploit in the next 30 days
percentile	float [0,1]	proportion of all scored vulnerabilities with the same or a lower EPSS score

summary of properties: cve, epss, percentile

EPSS Edges

EPSS - [:scores] -> CVE defined by EPSS.cve property, multiplicity 1